BOOX STORE JOURNEY

How the storefront, books, account, cart, and checkout work together

A reader arrives	Finds a book	Builds a cart	Pays securely

A code-based walkthrough of the project from the first browser request through verified payment.

CHAPTER ONE | THE USER JOURNEY
CHAPTER TWO | HOW THE BOOKS TRAVEL TO PURCHASE

Reading map

This guide follows the running application in the order a customer experiences it. It distinguishes what happens in the browser, what the server trusts, and what the payment provider confirms. File names are included so the explanation can be matched to the project.

Part	What it follows
Chapter 1 - The user journey	The React page, navigation, account, catalogue controls, book cards, cart state, and the customer-visible checkout return.
Chapter 2 - How books travel to purchase	Seed data and database, API request/response, cart payload, server-side price checks, order records, Stripe Checkout, webhook verification, and the final confirmation.
Trust and configuration	Where environment variables are loaded, why server secrets do not belong in the frontend, and what the app does not do after payment.
Code map	The main files to open when tracing a particular stage.

One sentence summary

The browser displays books and keeps a lightweight cart; the Express server owns authentication, database access, authoritative pricing, and order creation; Stripe hosts the payment form; and a signed webhook?not the browser redirect?changes an order to paid.

Scope note: this is an online checkout demonstration. The code records a paid order, but the reviewed project does not implement inventory deduction, shipping, tax calculation, email receipts, or physical/digital book fulfillment.

Chapter 1 - The user journey

1. The page arrives and builds the storefront

The HTML shell in frontend/index.html provides the root element and loads frontend/src/main.jsx. React mounts App inside that root. The app composition in frontend/src/App.jsx places the shared CartProvider around the page so header, book cards, and cart all read and update one cart. Tailwind styles are loaded from frontend/src/index.css.

```
createRoot(document.getElementById('root')).render(
  <StrictMode><App /></StrictMode>,
)
```

App loads the catalogue once on first render: fetch('/api/books'). While waiting, it shows a loading message; if the request fails, it reports an error instead of pretending that the catalogue is empty. An AbortController prevents a stale request from updating the component after it unmounts.

The page is arranged in a simple reading order: Header, Hero, AccountSection, BookGrid, CartSection, Events, About, and Footer. The header links jump to those sections rather than navigating to separate pages.

2. Finding a book

Header.jsx provides the Boox Store home link, section navigation, responsive mobile menu, and a live cart item count. On small screens the menu button toggles visibility; selecting a link closes it. Hero.jsx welcomes the reader and sends the Browse books button to the shop section. Events, About, and Footer provide informational sections; they are content, not separate APIs or booking flows.

BookGrid.jsx handles catalogue presentation. It starts with featured books; Browse all switches to the complete list. The category selector filters Fiction and Nonfiction. When browsing the full catalogue, it displays 20 books per page and resets the page when the view or category changes. These operations filter the loaded array in the browser; they do not issue a new request for each filter or page.

BookCard.jsx renders the cover, title, author, category, optional staff-pick label, price, and Add to cart control. Its count is derived from shared cart state. The image is decorative catalogue data; ordering uses the book ID, not an image URL.

3. Creating an account and staying signed in

AccountSection.jsx asks Better Auth for the current session with authClient.useSession(). When signed out, the reader can switch between signup and sign-in forms. Signup sends name, email, and password; login sends email and password. The form disables while submitting and surfaces an error if the request fails. When signed in, it shows the account name and email and provides sign-out.

frontend/src/lib/auth-client.js creates Better Auth?s browser client using the current site origin. Requests use the /api/auth path. In development the Vite proxy forwards /api to the Express server; in production the same Express app serves both the built frontend and API. Better Auth stores server-managed user/session information in PostgreSQL. Checkout separately checks the session on the server, so merely hiding or showing a UI button is not the security boundary.

Password length is constrained in the form and Better Auth is configured with a minimum of eight characters. The database auth schema defines user, session, account, and verification records. The project enables email/password authentication; no external login provider appears in the reviewed implementation.

Chapter 1 - The user journey (continued)

4. Building and keeping the cart

CartContext.jsx initializes cart state by reading localStorage under the key boox-store-cart. It validates restored entries, then exposes addItem, setItemQuantity, removeItem, clearCart, itemCount, and storageError through CartContext. Add increments an existing quantity or adds a new ID; quantity zero removes the item. Storage failures are logged and surfaced in the interface.

Only bookId and quantity are persisted. The cart is local to this browser and is not an account-synced basket. The displayed title, image, and unit price are joined back to the loaded book catalogue when CartSection renders. If a previously saved ID is no longer in the catalogue, the page warns the reader and offers removal.

QuantityStepper.jsx supplies plus/minus controls. CartSection.jsx shows the line totals and subtotal in USD, allows removing one line or emptying the cart, and disables checkout while books are loading, unavailable, or cart state is invalid for purchase. The subtotal is useful display feedback; the server computes the authoritative amount again.

```
// Local cart line shape sent to the server
{ bookId: 12, quantity: 2 }

// Stored by the browser: IDs and quantities only.
// Price and payable total are reloaded/calculated on the server.
```

5. Leaving for secure checkout and returning

Proceed to secure checkout POSTs JSON containing the cart ID/quantity pairs to /api/checkout/session. The browser does not send its subtotal as an amount to charge. A successful response contains a Stripe Checkout URL; the browser checks that the URL is HTTPS and belongs to checkout.stripe.com, then navigates away to Stripe-hosted payment.

If the reader cancels, Stripe returns to the store with checkout=cancel; the cart remains. On the success return, the URL also contains the Checkout Session ID. The client asks the store for the order status, retrying briefly while it is still pending. Only after the server reports paid does CartSection clear the cart and show the full-page confirmation, order number, and paid total. The reader chooses Continue browsing to remove checkout parameters and return to the book section.

This is deliberate: a redirect to a success URL is not proof of payment. The customer may arrive before the webhook is processed, so the interface can show a pending/check-again state. It also requires the same signed-in account to look up that order.

User-facing exceptions worth noting

The app reports catalogue loading errors, browser storage failures, unavailable saved books, authentication failures, checkout-start errors, and payment-status errors instead of showing a successful purchase by default. Cart contents survive a cancelled checkout or a failed payment.

Chapter 2 - How books travel to purchase

1. The catalogue enters the database

The canonical seed source is server/src/data/books.js. server/src/db/seed.js imports that array and inserts its records into PostgreSQL?s books table using Drizzle. Conflicting IDs are left alone, so running the seed again does not duplicate those IDs. The books table stores ID, title, author, category, numeric price, image path, staff-pick flag, and featured flag.

The frontend also has frontend/src/data/books.js, but the active page fetches /api/books and does not import that local array. The server-side seed data populates the database; PostgreSQL is the runtime catalogue used by this API. Images themselves remain static files under frontend/public/images; the database stores their paths.

2. From browser request to API response

React browser GET /api/books	Vite proxy in dev /api -> :3000	Express route GET /api/books	Drizzle -> PostgreSQL SELECT books

App.jsx calls the relative URL /api/books. During development, frontend/vite.config.js proxies /api to localhost:3000. In the server, src/index.js mounts Better Auth routes, then checkout routes, then health and book endpoints. GET /api/books selects the books table ordered by ID and serializes the result as JSON. The browser renders that JSON into BookCard components.

```
const allBooks = await db.select().from(books).orderBy(books.id)
res.json(allBooks)
```

In production, Express also serves frontend/dist as static assets. This lets the website and API use the same origin; the browser continues using relative /api URLs. /api/health is a simple server health response, not the catalogue endpoint.

3. A cart becomes a request, not an amount

CartSection posts only distinct book IDs and positive integer quantities. The backend route parses JSON with a 16 KB limit and calls parseCartItems. Validation rejects missing/empty lists, over 100 distinct books, duplicate IDs, non-integer or non-positive IDs/quantities, and excessive quantities. This prevents a caller from inventing a title or trusting a browser-supplied price.

```
POST /api/checkout/session
Content-Type: application/json

{
  "items": [
    { "bookId": 12, "quantity": 2 }
  ]
}
```

The server checks the authenticated session first. If there is no signed-in user, it returns 401. It then loads requested books from PostgreSQL. If a requested ID is missing, checkout stops with a clear unavailable-book response. Unit prices are read from the database, converted to integer cents, and multiplied by validated quantities; totals are checked for safe integer range. The server is authoritative for what gets charged.

Chapter 2 - From order to verified payment

4. Saving an order before payment

With trusted prices in hand, the server writes one pending row to orders and one row per book to order_items inside a database transaction. An order connects to its buyer and stores a total in cents. Each order item snapshots book ID, quantity, and unit price in cents. The order begins pending; a Stripe session ID is linked after Stripe creates the session.

The server then creates a Stripe Checkout Session with payment mode, customer email, order ID metadata, and line items derived from the database catalogue. It requests a success and cancellation return URL based on the configured store origin. When Stripe returns a session ID and URL, the server stores that ID on the pending order and returns the URL to the browser.

If creating/linking the Stripe session fails, the code tries to expire any created but unlinked Stripe session and delete the incomplete order. Errors are logged server-side and a safe message is returned to the browser.

5. Stripe payment and webhook confirmation

Browser redirects to Stripe Checkout	Stripe processes the payment	Signed webhook POST /api/stripe/webhook	Server verifies and updates order

Stripe sends the webhook to /api/stripe/webhook. This endpoint uses the raw request body because Stripe signature verification depends on the exact bytes received. It uses STRIPE_WEBHOOK_SECRET to verify the stripe-signature header. Requests with missing configuration or invalid signatures are rejected; a browser cannot simply claim a payment succeeded.

For the payment-completed events, the handler reads the order ID from Stripe session metadata, verifies that the stored Stripe session ID matches, requires Stripe?s payment_status to be paid, and compares the reported amount and currency with the stored order. Only then does it change a pending order to paid and set paidAt. Expired and asynchronous-failure events can set pending orders to expired or failed. Other event types are acknowledged without changing the order.

Conditional updates include the pending status and session ID, so repeat deliveries do not change an already-final order back to paid/failed. This is the intended idempotent state transition for normal repeated webhook delivery.

6. Browser confirms with the store

After redirect, the frontend calls GET /api/checkout/session/:sessionId. The route requires a signed-in user, validates the session ID shape, and queries by both Stripe session ID and user ID. It returns only the order ID, status, and total in cents. The client polls briefly if the webhook has not arrived yet, offers a manual retry, and shows success only for a server-reported paid status.

The resulting flow separates three claims: Stripe hosts the payment interface; the webhook proves the event and marks the database order; and the authenticated status endpoint tells the matching customer what the order state is.

Private configuration and trust boundaries

How the environment file is used privately

The real server/.env is a server-side configuration source. Backend modules import dotenv/config; Node loads those values into process.env when the server starts. Database connection code reads DATABASE_URL, Better Auth reads its secret and URL/origin settings, and checkout code reads Stripe?s secret and webhook signing secret. The server uses those values for its own database, auth, and Stripe operations. This walkthrough intentionally includes variable names and flow only; it does not include actual values from the private file.

server/.env.example is a template, not the live secret store. The root .gitignore ignores .env and .env.* while explicitly allowing .env.example. On hosting, these settings belong in the service?s private environment-variable settings rather than in browser code or committed source. The Vite frontend is not reading server/.env; no server secret should be copied to a VITE_* variable or embedded in a React bundle, because browser-delivered code is public.

Setting name	Purpose in this project
DATABASE_URL	Server-only PostgreSQL connection used by the postgres-js client and Drizzle.
BETTER_AUTH_SECRET	Server-side Better Auth signing/secret material.
BETTER_AUTH_URL / RENDER_EXTERNAL_URL	Server auth base URL; the hosted URL can take precedence.
CLIENT_URL / RENDER_EXTERNAL_URL	Trusted origin/store URL fallback for auth and checkout return links.
STRIPE_SECRET_KEY	Server-side Stripe API client used to create/expire Checkout Sessions.
STRIPE_WEBHOOK_SECRET	Server-only signing key used to verify webhook request authenticity.
PORT	Runtime listening port, with a local default in the server entry point.

The `.env` file is never needed by the browser. A customer sends cart IDs and quantities to the API; the server privately uses its database and Stripe credentials to determine the price and payment session.

What ?purchase complete? means here

A verified Stripe payment updates the order record and produces a customer confirmation. The reviewed implementation does not decrement stock, ship books, deliver downloads, calculate taxes/shipping, send a receipt, or provide an admin fulfillment workflow. The price display and checkout use USD, with no added tax or shipping. Treat the app as a learning/demo purchase pipeline unless fulfillment is added elsewhere.

Project code map

Stage	Primary files
Page startup and catalogue fetch	frontend/src/main.jsx; frontend/src/App.jsx
Sign-in and session	frontend/src/components/AccountSection.jsx; frontend/src/lib/auth-client.js; server/src/auth.js; server/src/db/auth-schema.js
Browse books and cart	frontend/src/components/BookGrid.jsx; BookCard.jsx; CartSection.jsx; QuantityStepper.jsx; frontend/src/context/CartContext.jsx
API and runtime	server/src/index.js; frontend/vite.config.js
Catalogue and persistence	server/src/data/books.js; server/src/db/seed.js; server/src/db/index.js; server/src/db/schema.js
Checkout, validation, webhook	server/src/checkout-routes.js; server/src/cart-validation.js; server/src/db/migrations/0001_create_orders.sql
Configuration safeguards	server/.env.example; .gitignore; server/drizzle.config.js

End of journey

A person chooses a book in the interface. The browser remembers which IDs and quantities they selected. The server retrieves the current catalogue price, creates a pending order, and asks Stripe to collect payment. Stripe?s signed webhook proves the outcome to the server. The store then reports the matching order status to the signed-in customer and clears the cart only after payment is confirmed.